

Portfolio Project 1:

Narrative of HR Attrition Data Analysis

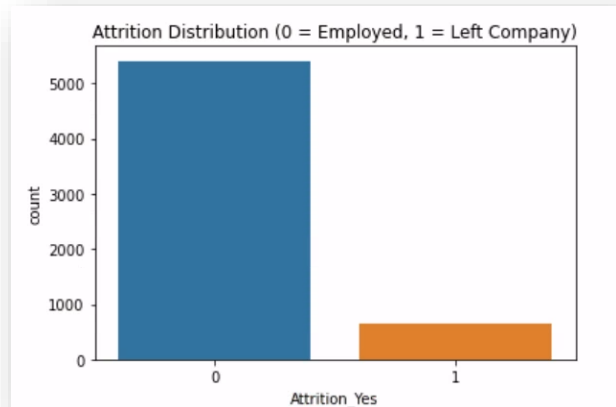
Tyrone Muhammad, BSDA, CPC

Project Origin Date: Sunday, December 15, 2024

Exploratory Data Analysis Findings

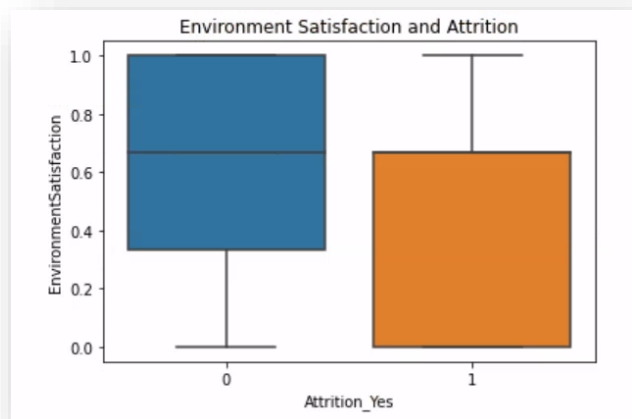
The Python code sampled below displays the number of employees who remained employed at the company compared to those who left the company. Members with the attribution of leaving the company are in orange while those who remain employed are attributed with blue.

```
# Attrition Trends:
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x='Attrition_Yes',
data=outer_merged_HR_data)
plt.title("Attrition Distribution (0 = Employed, 1 = Left Company)")
plt.show()
```



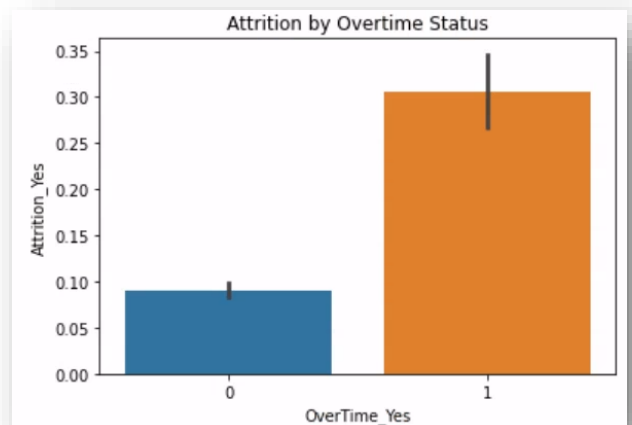
The Python code below permits the comparison of members satisfaction of the environment scores. Environment satisfaction surveys are displayed in blue for members who have remained with the company and the ratings are in orange for the members with the attrition of one who left the company. It is clear to see lower ratings come from those who leave the company.

```
# Environment Satisfaction vs. Attrition:
sns.boxplot(x='Attrition_Yes', y='EnvironmentSatisfaction',
data=outer_merged_HR_data)
plt.title("Environment Satisfaction and Attrition")
plt.show()
```



The next Python code shows the potential impact of overtime status. Its output displays the distribution of members who left compared to members who stayed and how they compared to receiving overtime. The offering of overtime does not retain the staff. Working beyond the regular shift overtime could relate to a non-satisfactory view of the work environment.

```
# Relationship Between OverTime_Yes and Attrition_Yes:
sns.barplot(x='OverTime_Yes', y='Attrition_Yes',
data=outer_merged_HR_data)
plt.title("Attrition by Overtime Status")
plt.show()
```



Predictive Model Features

In Project One, I successfully scaled and one-hot encoding my data set. In Project Two, I continued to prepare for predictive analytics by doing a correlation analysis and picking from the top 10 results of those who left the company (attrition equals “yes”).

```
# Correlation Analysis: Identify features most
correlated with Attrition_Yes
corr = outer_merged_HR_data.corr()
attrition_corr =
corr['Attrition_Yes'].sort_values(ascending=False)
print(attrition_corr.head(10)) # Top features
correlated with attrition
```

```
Attrition_Yes      1.000000
Over18_Yes         0.427143
OverTime_No        0.333817
Gender_M           0.304674
MaritalStatus_Single 0.295619
Gender_F           0.271827
BusinessTravel_Travel_Frequently 0.264664
DistanceFromHome   0.264633
Department_Sales   0.241889
MaritalStatus_Married 0.241205
Name: Attrition_Yes, dtype: float64
```

Next, I selected from these top 10 features who are correlated with Attrition_Yes to be utilized within my predictive modeling. Lastly, in this syntax I confirm the results. The same syntax was utilized for a revised version which displays less features for maintaining accuracy but allowing more focused suggestions for a better work environment and greater employee retention.

```
# Selected features from previous Top 10 features correlated with attrition
selected_features = [
    'Over18_Yes','OverTime_No','Gender_M','MaritalStatus_Single',
    'Gender_F','BusinessTravel_Travel_Frequently',
    'DistanceFromHome','Department_Sales','MaritalStatus_Married'
]

# Create feature (X) and target (y) datasets
X = outer_merged_HR_data[selected_features]
y = outer_merged_HR_data['Attrition_Yes']

# Verify shapes
print(f"Features shape: {X.shape}, Target shape: {y.shape}")
```

```
# Selected features from previous Top 10 features correlated with attrition
selected_features = [
    'Over18_Yes', 'OverTime_No', 'Gender_M', 'MaritalStatus_Single',
    'Gender_F', 'BusinessTravel_Travel_Frequently',
    'DistanceFromHome', 'Department_Sales', 'MaritalStatus_Married'
]

# Create feature (X) and target (y) datasets
X = outer_merged_HR_data[selected_features]
y = outer_merged_HR_data['Attrition_Yes']

# Verify shapes
print(f"Features shape: {X.shape}, Target shape: {y.shape}")
```

```
Features shape: (6040, 9), Target shape: (6040,)
```

Predictive Model Results

As discussed in assignment 5-1 Applying Predictive Models, I have decided to utilize logistics regression as well as a random forest model to analyze the HR attrition data. Random forests are ensemble models combining multiple decision trees (Muhammad, 2024). Logistic regression predicts binary outcomes, like a simple yes/no. Below is the syntax utilized as well as an image of the outputs.

Logistic Regression Model:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Logistic Regression Model
log_model = LogisticRegression()
log_model.fit(X_train, y_train)

# Predictions and Evaluation
y_pred_log = log_model.predict(X_test)

print("Logistic Regression Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:\n", confusion_matrix(y_test,
y_pred_log))
print("Classification Report:\n",
classification_report(y_test, y_pred_log))
```

Logistic Regression Results:					
Accuracy: 0.8896247240618101					
Confusion Matrix:					
[[1599 11]					
[189 13]]					
Classification Report:					
	precision	recall	f1-score	support	
0	0.89	0.99	0.94	1610	
1	0.54	0.06	0.12	202	
accuracy			0.89	1812	
macro avg	0.72	0.53	0.53	1812	
weighted avg	0.85	0.89	0.85	1812	

Random Forest Model

```
from sklearn.ensemble import RandomForestClassifier

# Random Forest Model
rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
rf_model.fit(X_train, y_train)

# Predictions and Evaluation
y_pred_rf = rf_model.predict(X_test)

print("Random Forest Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test,
y_pred_rf))
print("Classification Report:\n",
classification_report(y_test, y_pred_rf))
```

Random Forest Results:					
Accuracy: 0.8719646799116998					
Confusion Matrix:					
[[1528 82]					
[150 52]]					
Classification Report:					
	precision	recall	f1-score	support	
0	0.91	0.95	0.93	1610	
1	0.39	0.26	0.31	202	
accuracy			0.87	1812	
macro avg	0.65	0.60	0.62	1812	
weighted avg	0.85	0.87	0.86	1812	

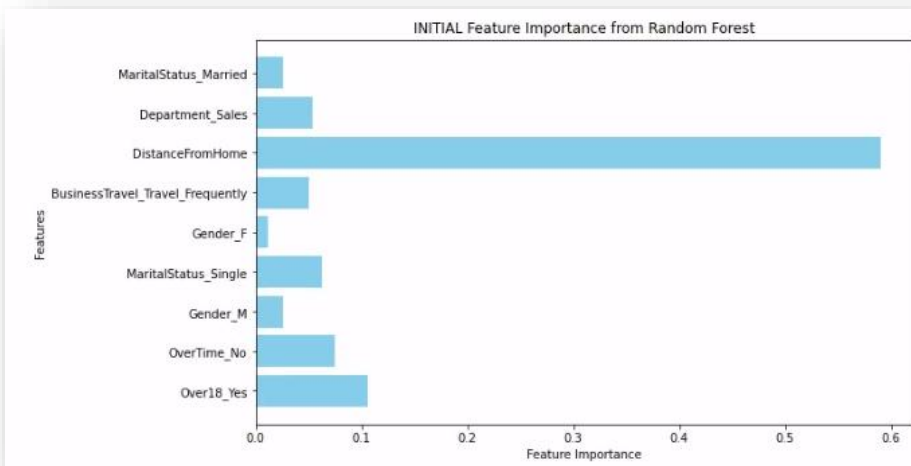
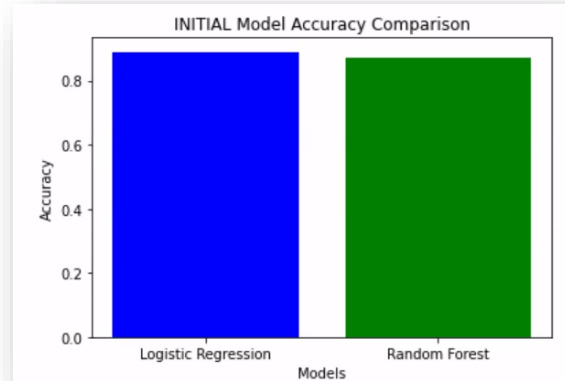
Accuracy of Outcomes

The **INITIAL** accuracy of the two models is only moderately high. For both models the accuracy scores as well as weighted average scores were above 0.85. There is a consistency of highly accurate information of the features selected showing that exploring this could lead to better ways to retain employees.

```
import matplotlib.pyplot as plt
```

```
# Accuracy scores
accuracy_scores = [accuracy_score(y_test, y_pred_log),
accuracy_score(y_test, y_pred_rf)]
model_names = ['Logistic Regression', 'Random Forest']

plt.bar(model_names, accuracy_scores, color=['blue', 'green'])
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.title('INITIAL/REVISED Model Accuracy Comparison')
plt.show()
```



```
importances =
rf_model.feature_importances_

features = X.columns

plt.figure(figsize=(10, 6))
plt.barh(features, importances,
color='skyblue')
plt.xlabel('Feature Importance')
plt.ylabel('Features')
plt.title('INITIAL/REVISED Feature
Importance from Random Forest')
plt.show()
```

Below are **REVISED** results with “noise” or features removed while still maintaining model accuracy. Removing certain features such as gender and marital status permit the company to focus on elements they can control within the workplace.

